

Portale Urbanistico Comunale

Architettura Multi-Indice a Grafo Ortogonale per la Gestione di
Disservizi Territoriali

Relazione Tecnica di Progettazione — Corso di PSD 2025–2026
Sviluppo Autonomo Individuale — Traccia 3

Autore: Salvatore Pisu ([sandie69p](#))
Repository: github.com/sandie69p/Progetto-PSD-2025-2026--Traccia-3
Ambiente: Arch Linux TTY — GCC
Standard: C99 Compilato con GNU Make
Validazione: Valgrind Memcheck & GDB Subsystem

Indice

1	Introduzione al progetto	4
1.1	Approccio metodologico alla Traccia 3	4
1.2	Vincoli strutturali e limiti delle soluzioni lineari	4
1.3	Transizione al paradigma multi-indice	4
1.4	Consolidamento architetturale e risultato finale	4
2	Motivazione della scelta dell'ADT	5
2.1	Definizione dell'ADT adottato	5
2.2	Requisiti funzionali del dominio applicativo	5
2.3	Analisi comparativa delle strutture dati candidate	5
2.4	Giustificazione della scelta ottimale	6
3	Manuale d'uso e sistema di build	6
3.1	Automazione del ciclo di build tramite GNU Make	6
3.1.1	Target <code>clean</code> : sanificazione dell'albero dei sorgenti	6
3.1.2	Target <code>all</code> : compilazione del binario principale	6
3.1.3	Target <code>run</code> : esecuzione nativa dell'applicazione	7
3.1.4	Target <code>debug</code> : analisi con GNU Debugger	7
3.1.5	Target <code>valgrind</code> : analisi dinamica della memoria	7
3.1.6	Target <code>test</code> : esecuzione della suite di unit testing	7
3.2	Funzionalità operative della dashboard	7
3.2.1	Comando (1) — Inserimento nuova segnalazione	7
3.2.2	Comando (2) — Rimozione segnalazione per ID	7
3.2.3	Comando (3) — Visualizzazione prime 20 segnalazioni per data	7
3.2.4	Comando (4) — Ricerca live incrementale per ID o categoria	8
3.2.5	Comando (5) — Modifica stato operativo di una segnalazione	8
3.2.6	Comando (6) — Visualizzazione descrizione estesa di un record	8
3.2.7	Comando (0) — Salvataggio su disco, generazione report e uscita	8
4	Architettura dei moduli e organizzazione del repository	9
4.1	Cartella radice del progetto	9
4.1.1	File <code>LICENSE</code>	9
4.1.2	File <code>README.md</code>	9
4.1.3	File <code>.gitignore</code>	9
4.1.4	Rimando al Makefile	9
4.2	Cartella <code>docs/</code>	9
4.2.1	File <code>Doxyfile</code>	9
4.2.2	Documentazione in <code>L^AT_EX</code>	9
4.3	Cartella <code>src/</code>	9
4.3.1	File <code>main.c</code>	10
4.3.2	File <code>report_comune.txt</code>	10

4.3.3	Cartella <code>test/</code> e file <code>main.t.c</code>	10
4.4	Cartella <code>components/</code>	10
4.4.1	File <code>adt/struct.c</code>	10
4.4.2	File <code>adt/struct.h</code>	10
4.4.3	File <code>HUD/hud.c</code>	10
4.4.4	File <code>HUD/hud.h</code>	10
4.4.5	File <code>database/database.bin</code>	10
5	Strategia di eliminazione dei memory leak	11
5.1	Analisi dinamica con Valgrind Memcheck	11
5.1.1	Risoluzione dei difetti rilevati	11
5.1.2	<code>calloc</code> vs <code>malloc</code> : motivazione della scelta	11
5.2	Debugging a runtime con GNU Debugger	12
6	Specifica sintattica e semantica	12
6.1	Specifica sintattica di <code>hud.c</code>	12
6.2	Specifica semantica di <code>hud.c</code>	12
6.2.1	<code>dashboard</code>	12
6.2.2	<code>insertNewSeg</code>	13
6.2.3	<code>removeSeg</code>	13
6.2.4	<code>showSeg</code>	13
6.2.5	<code>init_search_seg</code>	13
6.2.6	<code>modifySegHud</code>	13
6.2.7	<code>showDescription</code>	14
6.2.8	<code>salvataggio</code>	14
6.2.9	<code>intestation</code> (interna)	14
6.2.10	<code>getReport</code> (interna)	14
6.2.11	<code>getSeg</code> (interna)	14
6.3	Specifica sintattica di <code>struct.c</code>	15
6.4	Specifica semantica di <code>struct.c</code>	16
6.4.1	<code>init_root</code>	16
6.4.2	<code>init_loadingDb</code>	16
6.4.3	<code>init_sorting</code>	16
6.4.4	<code>init_removeSeg</code>	17
6.4.5	Getter globali: <code>getTotalSeg</code> , <code>getTotalAperte</code> , <code>getTotalRis</code> , <code>getTotalChiuse</code> , <code>getMostUrgenti</code> , <code>getDataHead</code>	17
6.4.6	<code>getSegCountByCategory</code>	17
6.4.7	<code>getMaxCat</code>	17
6.4.8	<code>getMaxCatName</code>	18
6.4.9	Getter atomici: <code>getID</code> , <code>getName</code> , <code>getDescription</code> , <code>getCat</code> , <code>getRawData</code> , <code>getUrg</code> , <code>getState</code>	18
6.4.10	<code>getData</code>	18
6.4.11	Iteratori: <code>nextForID</code> , <code>nextForData</code> , <code>nextForUrg</code>	18

6.4.12	<code>getNewSeg</code>	18
6.4.13	<code>save_records</code>	19
6.4.14	<code>deleteGraph</code>	19
6.4.15	<code>search_seg</code>	19
6.4.16	<code>modifySeg</code>	19
6.4.17	<code>findSegById</code>	20
6.4.18	<code>getCategoryIndex</code> (interna)	20
6.4.19	<code>insertOnGraph</code> (interna)	20
6.4.20	<code>quicksort</code> (interna)	21
6.4.21	<code>getRandomId</code> (interna)	21
6.4.22	<code>getPosition</code> (interna)	21
6.4.23	<code>appendNewSeg</code> (interna)	21
6.5	Specifica sintattica di <code>main.c</code>	22
6.6	Specifica semantica di <code>main.c</code>	22
6.6.1	<code>init</code>	22
6.6.2	<code>handle_signal</code>	22
7	Suite di unit testing: <code>main.t.c</code>	22
8	Ambiente di sviluppo e toolchain	25
8.1	Sistema operativo	25
8.2	Compilatore	25
8.3	Standard C	25
8.4	Flag di compilazione	25
8.5	Sistema di build	26
8.6	Analisi dinamica della memoria	26
8.7	Analisi di debug a runtime	26

1. Introduzione al progetto

1.1. Approccio metodologico alla Traccia 3

L’approccio iniziale alla Traccia 3 (“Portale Urbanistico Comunale”) è stato guidato dalla volontà di elaborare una soluzione nativa, cruda e totalmente autocontenuta. Rifiutando l’impiego di database relazionali esterni, motori di persistenza di terze parti o astrazioni preconfezionate, ho scelto di operare direttamente a basso livello sulla memoria Heap del calcolatore in C99. L’obiettivo era creare un sistema transazionale in grado di catalogare migliaia di record senza alcun overhead grafico, massimizzando le prestazioni computazionali e l’efficienza nell’utilizzo dei registri della CPU.

1.2. Vincoli strutturali e limiti delle soluzioni lineari

La principale complessità della traccia risiedeva nell’obbligo di garantire l’interrogazione e la navigazione dei dati secondo quattro dimensioni logiche totalmente indipendenti e asincrone (ID categoria, data cronologica, stato operativo e livello di urgenza).

Nei primi stadi di sviluppo, ho tentato di risolvere il problema utilizzando strutture dati lineari isolate. Ho provato ad implementare un array statico di record e, successivamente, una singola lista concatenata. Questo approccio ha evidenziato limiti strutturali invalidanti: ordinare la memoria per soddisfare una dimensione di consultazione (ad esempio, la cronologia) distruggeva istantaneamente l’ordinamento richiesto per le altre dimensioni. Per effettuare le ricerche incrementali e i filtri dinamici, il sistema si trovava costretto a eseguire continue scansioni sequenziali esaustive con complessità temporale pari a $\mathcal{O}(N)$. Questo generava colli di bottiglia prestazionali e un ammasso di codice ridondante e instabile.

1.3. Transizione al paradigma multi-indice

Preso atto dei limiti delle strutture lineari semplici, ho effettuato un refactoring radicale del paradigma logico. Ho compreso che il record (la segnalazione urbana) doveva esistere come entità fisica **unica in tutta la memoria Heap**, ma che le relazioni di attraversamento dovevano essere scorrelate dal dato stesso. Sono passato dal concetto di “lista ordinata di elementi” al concetto di **grafo multi-indice a puntatori ortogonali**. Separando la persistenza del record fisico dalla topologia dei vettori di navigazione, ho rimosso la necessità di duplicare la memoria, gettando le basi per un’architettura in cui l’accesso alle teste degli indici principali avviene in tempo costante $\mathcal{O}(1)$.

1.4. Consolidamento architetturale e risultato finale

Per consolidare questo nuovo modello, ho blindato l’intero nucleo algoritmico applicando in modo tassativo il principio dell’Information Hiding tramite *opaque pointer*. Ho rimosso qualsiasi definizione di struttura dai file header, confinando i tipi fisici all’interno del file di implementazione dell’ADT. Ho scritto un motore di ordinamento ottimizzato basato su

Quicksort applicato a vettori di puntatori temporanei per la ricostruzione iniziale degli indici e ho implementato sbarramenti difensivi per la cattura dell'input a basso livello da terminale (POSIX). Il risultato finale è un ecosistema software minimale e simmetrico, progettato per ridurre il rischio di corruzioni e memory leak, e verificato a tal fine con strumenti di analisi dinamica come Valgrind.

2. Motivazione della scelta dell'ADT

2.1. Definizione dell'ADT adottato

L'Abstract Data Type sviluppato si configura come un **Grafo Multi-Indice Ordinato con Puntatori Ortogonali ed Incapsulamento Opaco**. La struttura di controllo centrale (**Root**) mantiene i descrittori e i puntatori di testa verso quattro assi di navigazione indipendenti, mentre i singoli nodi atomici contengono al proprio interno i campi dato e il set di puntatori necessari a innestarsi contemporaneamente in tutte le catene logiche.

2.2. Requisiti funzionali del dominio applicativo

Un portale urbanistico comunale orientato alla consultazione intensiva richiede che le operazioni di filtraggio statistico, la visualizzazione cronologica delle priorità e la ricerca live per ID avvengano con latenza minima. Se un operatore richiede la visualizzazione delle segnalazioni più urgenti, il sistema non deve scansionare l'intero database: accede direttamente alla testa del bucket di urgenza, già organizzato a tempo di inserimento. I requisiti del problema impongono quindi: zero ridondanza dei dati, persistenza immutabile dell'ID e multidimensionalità degli accessi.

2.3. Analisi comparativa delle strutture dati candidate

Per giungere alla scelta ottimale, ho analizzato e scartato sistematicamente le strutture dati classiche:

- **Array statico / dinamico:** Consentirebbe la ricerca binaria in $\mathcal{O}(\log N)$ per un singolo asse ordinato, ma gli inserimenti e le rimozioni richiederebbero lo shifting dei blocchi di memoria ($\mathcal{O}(N)$). Non supporta la navigazione multi-assiale simultanea senza duplicare l'intero array per quattro volte.
- **Lista concatenata singola:** Garantisce l'allocazione dinamica, ma la ricerca e l'ordinamento degradano a $\mathcal{O}(N)$. Un solo puntatore **next** vincola il sistema a un unico percorso topologico piatto.
- **Stack (Pila) / Queue (Coda) singola:** Strutture ad accesso restrittivo (LIFO/FIFO). Inutilizzabili per un sistema che richiede l'accesso casuale, la cancellazione puntuale di record intermedi e la ricerca live su stringa parziale.
- **Alberi Binari di Ricerca (BST):** Un BST offrirebbe ricerche in $\mathcal{O}(\log N)$ nel caso medio, ma in assenza di bilanciamento nativo rischia di degenerare in una lista con

complessità $\mathcal{O}(N)$. Gestire quattro alberi distinti avrebbe comportato la duplicazione dei nodi o un intreccio di puntatori padri/figli complesso e propenso a errori di segmentazione.

- **Array di liste:** Utile per raggruppare i record per categoria, ma isola i dati impedendo una visione cronologica o di urgenza globale rapida senza algoritmi di fusione aggiuntivi.
- **Hash Table:** Eccellente per la ricerca puntuale dell’ID in tempo medio $\mathcal{O}(1)$, ma inadatta quando viene richiesto l’attraversamento ordinato per data o la visualizzazione delle prime N segnalazioni, poiché l’hashing distrugge nativamente qualsiasi relazione d’ordine tra le chiavi.

2.4. Giustificazione della scelta ottimale

Nel contesto specifico della traccia, il grafo multi-indice a puntatori ortogonali risulta più adatto rispetto alle alternative analizzate. Mantenendo il record fisicamente unico in memoria Heap, azzerava lo spreco di RAM. I quattro puntatori interni al nodo agiscono come “binari” indipendenti: l’accesso alle teste degli indici principali (stato, urgenza, categoria) avviene in tempo costante $\mathcal{O}(1)$, mentre la scansione parziale su una lista logica già filtrata riduce lo spazio di ricerca rispetto a una scansione globale. L’inserimento ordinato a caldo sconta un costo lineare sulla lista interessata, ma stabilizza l’intera infrastruttura garantendo letture e modifiche immediate, rispondendo ai vincoli ingegneristici del progetto.

3. Manuale d’uso e sistema di build

3.1. Automazione del ciclo di build tramite GNU Make

L’automazione del ciclo di build, test e monitoraggio è interamente delegata a un **Makefile** rigido, configurato per imporre standard di compilazione severi ed evitare l’introduzione di warning o codice non conforme.

3.1.1. Target **clean**: sanificazione dell’albero dei sorgenti

Il comando **make clean** rimuove ricorsivamente la cartella degli oggetti intermedi (**src/components/object**) i binari generati del sistema principale e l’eseguitibile della suite di test. Garantisce che ogni successiva compilazione avvenga partendo da uno stato pulito (Tabula Rasa), escludendo l’uso di vecchi file oggetto non aggiornati.

3.1.2. Target **all**: compilazione del binario principale

Il comando **make** (target di default **all**) avvia la compilazione dell’applicazione principale. Crea la directory degli oggetti, compila in modo indipendente **main.c**, **hud.c** e **struct.c** applicando i flag **-Wall -Wextra -std=c99 -O2 -g**, e infine esegue il linking dei file oggetto per produrre l’eseguitibile finale **src/municipal_system**.

3.1.3. Target run: esecuzione nativa dell'applicazione

Il comando `make run` richiama il target di compilazione per accertarsi che il binario sia aggiornato e successivamente esegue nativamente l'applicazione all'interno del terminale, agganciando automaticamente il database binario di produzione situato nel path corretto.

3.1.4. Target debug: analisi con GNU Debugger

Il comando `make debug` compila i sorgenti disabilitando le ottimizzazioni aggressive del compilatore che potrebbero mascherare lo stato dei registri (rimuove `-O2`) e invoca direttamente il debugger GNU (`gdb`) caricando il binario con i simboli di debug completi per l'analisi immediata dello stack dei record.

3.1.5. Target valgrind: analisi dinamica della memoria

Il comando `make valgrind` esegue il binario principale sotto la supervisione del tool di analisi dinamica Memcheck, configurato con i flag `-leak-check=full -show-leak-kinds=all -track-origins=yes` per intercettare l'utilizzo di variabili non inizializzate o mancate deallocazioni della memoria Heap.

3.1.6. Target test: esecuzione della suite di unit testing

Il comando `make test` compila l'infrastruttura di unit testing isolata scritta nel file `src/test/main.t.c`, effettuando il linking con l'oggetto dell'ADT `struct.o`. Subito dopo la compilazione, l'eseguibile di test viene lanciato automaticamente sotto Valgrind, validando la stabilità logica dell'ADT prima del deployment nell'HUD principale.

3.2. Funzionalità operative della dashboard

La dashboard dell'HUD modella un'interfaccia geometrica rigorosa a 101 caratteri di larghezza, incorniciata da caratteri speciali ANSI per l'allineamento visivo in TTY nativa.

3.2.1. Comando (1) — Inserimento nuova segnalazione

Consente l'immissione di un nuovo record nel comune. L'operatore seleziona la macro-categoria; il sistema genera in automatico un ID a 7 cifre protetto da collisioni e acquisisce il timestamp orario. Un ciclo difensivo esegue lo *Stream Masking* svuotando il buffer di input tramite `getchar()` per impedire che stringhe eccedenti i limiti della `struct` sporchino le successive acquisizioni. Il record viene iniettato in tutti e quattro gli indici del grafo.

3.2.2. Comando (2) — Rimozione segnalazione per ID

Richiede l'ID numerico del disservizio da eliminare. Se l'utente digita 0, scatta una via di fuga immediata. Qualora venga inserito un ID valido, la funzione scollega il nodo modificando i puntatori dei nodi adiacenti su tutti e quattro gli assi e libera fisicamente la memoria Heap tramite `free()`.

3.2.3. Comando (3) — Visualizzazione prime 20 segnalazioni per data

Interroga l'indice cronologico globale partendo dal puntatore `dataNode` della Root. Sfruttando la pre-organizzazione del grafo, la funzione esegue un attraversamento lineare

scorrendo il puntatore `nextData` per i primi 20 nodi, stampando a video una tabella geometrica pulita e simmetrica.

3.2.4. Comando (4) — Ricerca live incrementale per ID o categoria

Rappresenta il modulo più avanzato dell'interfaccia uomo-macchina. Il terminale viene forzato in modalità non canonica (`stty -icanon -echo`). Il sistema cattura i singoli caratteri digitati senza attendere il tasto Invio. Ad ogni digitazione, pulisce lo schermo e filtra il grafo: se i primi caratteri corrispondono a un codice categoria valido, la ricerca si restringe al singolo bucket. Il tasto `ESC` interrompe la ricerca live e ripristina la modalità normale (`stty cooked echo`).

3.2.5. Comando (5) — Modifica stato operativo di una segnalazione

Permette la transizione di stato di un record (Aperta → In Risoluzione → Chiusa). L'operatore immette l'ID e seleziona il nuovo codice stato. Se viene premuto `ESC`, la funzione intercetta il codice ASCII 27 ed esegue un *rollback* totale lasciando inalterati puntatori e contatori di stato.

3.2.6. Comando (6) — Visualizzazione descrizione estesa di un record

Dato che l'HUD non può accedere direttamente ai campi della struttura a causa dell'Information Hiding, questa opzione richiede l'ID, interroga la funzione estrattiva pubblica dell'ADT e stampa in un box geometrico dedicato il buffer testuale completo da 1024 byte della descrizione.

3.2.7. Comando (0) — Salvataggio su disco, generazione report e uscita

Consolida lo stato del sistema: apre il file `database.bin` in modalità distruttiva ("`wb`"), scrive in modo sequenziale i record seguendo l'indice cronologico, genera un report statistico in formato testuale (`report_comune.txt`), invoca la distruzione controllata del grafo azzerando la RAM e arresta il processo in modo deterministico.

4. Architettura dei moduli e organizzazione del repository

4.1. Cartella radice del progetto

La cartella radice del progetto racchiude i file di configurazione, automazione e tracciamento del codice sorgente.

4.1.1. File `LICENSE`

Il file `LICENSE` contiene il testo integrale della licenza **GNU Affero General Public License versione 3 (AGPL-3.0)**. Questa scelta protegge il software garantendo che chiunque modifichi o utilizzi l'architettura sia obbligato a rilasciare il codice sorgente completo sotto la stessa licenza, preservando il valore dell'indipendenza e dell'open-source.

4.1.2. File `README.md`

Il file `README.md` descrive l'architettura generale del sistema, riporta le istruzioni rapide per la compilazione tramite `Makefile`, elenca i dettagli di implementazione del grafo multi-indice e riporta il log di Valgrind per certificare l'assenza di leak prima della sottomissione del progetto.

4.1.3. File `.gitignore`

Il file `.gitignore` impedisce il tracciamento su Git dei file binari generati, dei file oggetto intermedi (`.o`) prodotti dal compilatore e dei database locali temporanei utilizzati durante lo stress-testing, mantenendo pulita la repository ufficiale su GitHub.

4.1.4. Rimando al `Makefile`

La radice ospita inoltre il `Makefile` principale del progetto, strutturato per centralizzare la gestione dei path e delle dipendenze dei file. Per la descrizione estesa dei singoli target di compilazione e dei flag di ottimizzazione si rimanda alla Sezione 3.1.

4.2. Cartella `docs/`

La directory deputata alla documentazione e alle specifiche formali del sistema.

4.2.1. File `Doxyfile`

Il file `Doxyfile` contiene i parametri di configurazione per il tool **Doxygen**, strutturato per scansionare i tag speciali inseriti nei commenti dei sorgenti (`@brief`, `@param`, `@pre`, `@post`) e generare in automatico la documentazione HTML delle API pubbliche dell'ADT.

4.2.2. Documentazione in $\text{\LaTeX}$

Contiene i file sorgenti e gli stili tipografici necessari alla compilazione di questa stessa relazione tecnica, strutturata per formalizzare la semantica del software in modo rigoroso.

4.3. Cartella `src/`

Il cuore logico dell'applicazione, suddiviso nei file di bootstrap, reportistica e testing.

4.3.1. File `main.c`

Rappresenta l'entry point del programma. Inizializza il gestore asincrono dei segnali, invoca il bootstrap della radice richiamando `init_root()` e lancia il ciclo infinito (`while(1)`) che coordina il flusso alternato tra il rendering della dashboard e la cattura selettiva dei comandi dell'operatore.

4.3.2. File `report_comune.txt`

È il file di report statistico generato automaticamente ad ogni chiusura pulita del programma (opzione 0). Esporta su disco il volume totale delle segnalazioni attive, la ripartizione per stato e livello di urgenza e indica l'area urbana che registra il maggior tasso di criticità.

4.3.3. Cartella `test/` e file `main.t.c`

Directory isolata che ospita la suite di test atomica `main.t.c`. Questo file implementa una serie di test automatizzati indipendenti dall'HUD, scritti per verificare la correttezza logica delle funzioni primitive dell'ADT mediante asserzioni strutturali.

4.4. Cartella `components/`

La sottocartella architetturale in cui risiedono i moduli strutturali e i file di persistenza.

4.4.1. File `adt/struct.c`

È il file di implementazione dell'Abstract Data Type. Contiene le definizioni fisiche segrete delle strutture `struct segnalazione` e `struct root`, i meccanismi di allocazione dinamica, gli algoritmi di partizionamento Quicksort per il riallineamento dei vettori ortogonali e le routine di lettura/scrittura dei blocchi binari.

4.4.2. File `adt/struct.h`

L'interfaccia pubblica dell'ADT. Esporta unicamente i tipi opachi e i prototipi delle funzioni di consultazione e mutazione. Non rivela alcun dettaglio sui campi interni o sui puntatori del grafo, garantendo il completo disaccoppiamento del codice (Information Hiding).

4.4.3. File `HUD/hud.c`

Il Presentation Layer dell'applicazione. Implementa le funzioni di rendering tabellare, gestisce i banner simmetrici a 101 caratteri e converte l'input dell'utente in chiamate transazionali verso l'ADT.

4.4.4. File `HUD/hud.h`

L'header che espone le funzioni grafiche dell'interfaccia testuale. Permette al `main.c` di invocare la dashboard e le schermate di acquisizione dati senza che il bootstrap conosca le routine di manipolazione del terminale.

4.4.5. File `database/database.bin`

Il file di persistenza su disco. Memorizza le segnalazioni sotto forma di record binari a dimensione fissa (struttura serializzata da 1168 byte per blocco), consentendo il caricamento e il salvataggio tramite operazioni di blocco I/O sequenziali.

5. Strategia di eliminazione dei memory leak

5.1. Analisi dinamica con Valgrind Memcheck

La natura multidimensionale del grafo ortogonale — in cui uno stesso blocco di memoria Heap è agganciato contemporaneamente da quattro puntatori diversi — ha reso la gestione della deallocazione una sfida ingegneristica complessa. Durante le prime build del progetto, liberare la memoria causava crash o lasciava blocchi orfani nell'Heap.

5.1.1. Risoluzione dei difetti rilevati

Attraverso un processo di debugging ricorsivo guidato dai log di Valgrind, ho analizzato ogni singola segnalazione di errore:

- **Invalid Read / Write of size X:** Identificava tentativi dell'HUD di accedere a nodi già deallocati da una precedente rimozione. Ho risolto introducendo sbarramenti rigidi con controllo di uguaglianza a `NULL` e forzando l'aggiornamento immediato di tutti e quattro i puntatori adiacenti durante lo sfilamento del nodo.
- **Double Free Error:** Valgrind evidenziava che scorrere le liste in modo casuale portava a liberare lo stesso nodo più volte. Per sanare questo bug strutturale, ho progettato la funzione `deleteGraph` affinché effettui la demolizione controllata scorrendo **esclusivamente l'asse cronologico** (`nextData`). Dato che ogni record compare una e una sola volta nella catena cronologica globale, la funzione libera il nodo in sicurezza e azzerava i puntatori dei descrittori di testa della Root, garantendo la pulizia totale.

5.1.2. `calloc` vs `malloc`: motivazione della scelta

Nelle prime fasi di sviluppo, l'utilizzo della classica `malloc` per l'allocazione delle strutture di controllo (Root) e dei descrittori dei bucket introduceva errori intermittenti di lettura estranea. Valgrind segnalava costantemente il flag `"Conditional jump or move depends on uninitialised value(s)"`.

Questo accadeva perché la `malloc` alloca la memoria senza pulirla, lasciando nei puntatori `next` residui di bit casuali presenti nell'Heap della macchina. Quando il sistema tentava di navigare il grafo, scambiava quella “spazzatura” per indirizzi di memoria validi, causando violazioni di accesso. Ho sostituito le allocazioni critiche introducendo la `calloc`, che non solo riserva lo spazio necessario ma azzerava bit a bit l'intero blocco allocato, garantendo che ogni puntatore non ancora esplicitamente assegnato nasca nativamente impostato a `NULL`. Questo sbarramento preventivo ha azzerato i jump condizionali errati e ha stabilizzato l'architettura.

5.2. Debugging a runtime con GNU Debugger

Nei casi in cui il programma andava incontro a blocchi improvvisi o anomalie logiche durante la manipolazione in tempo reale dei descrittori, ho utilizzato il debugger GNU (gdb). Impostando i breakpoint all'interno di `modifySeg` e ispezionando lo stato dei registri tramite il comando `print root->stato->aperto->head`, sono riuscito a mappare visivamente lo sfilamento dei nodi dalle catene ortonormali, correggendo gli algoritmi di inserimento in testa e assicurando la coerenza dei flussi di memoria.

6. Specifica sintattica e semantica

Le specifiche formalizzano il comportamento di ciascun modulo eliminando qualsiasi ambiguità implementativa. Vengono elencate tutte le funzioni presenti nei file sorgente, distinguendo le funzioni pubbliche esportate dall'interfaccia da quelle interne statiche, che rimangono confinate al solo modulo di implementazione.

6.1. Specifica sintattica di `hud.c`

Le funzioni **pubbliche** esportate da `hud.h` sono:

```
void dashboard(Root sistema);
void insertNewSeg(Root root);
void removeSeg(Root root);
void showSeg(Root root);
void init_search_seg(Root root);
void modifySegHud(Root root);
void showDescription(Root root);
void salvataggio(Root sistema);
```

Le funzioni **interne statiche** (confinare in `hud.c`) sono:

```
static void intestation(void);
static void getReport(Root sistema);
static void getSeg(s node);
```

6.2. Specifica semantica di `hud.c`

6.2.1. `dashboard`

Pre-condizioni: `sistema` deve essere un puntatore valido a una struttura `Root` inizializzata (non nullo).

Post-condizioni: Il terminale viene ripulito e il pannello di controllo viene renderizzato sullo standard output. Lo stato della RAM e del grafo rimane invariato.

Effetti collaterali: Emette i flussi di testo formattati a 101 caratteri su `stdout`, inviando esplicitamente il flush allo stream.

6.2.2. `insertNewSeg`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: Il terminale viene ripulito e la cornice dell'HUD viene renderizzata. Il controllo dei puntatori viene trasferito alla routine di acquisizione `getNewSeg()`. Lo stato della RAM viene modificato solo in base all'esito della funzione delegata.

Effetti collaterali: Delega l'intera logica di inserimento, allocazione e persistenza a `getNewSeg()`.

6.2.3. `removeSeg`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: Se l'ID è conforme e presente nel database, il nodo viene rimosso fisicamente dai quattro indici paralleli e la memoria viene deallocata; in caso contrario, la RAM non subisce alterazioni. Il terminale viene aggiornato graficamente prima del rientro alla dashboard.

Effetti collaterali: Validazione multi-stadio dell'input tramite *Stream Masking*. Confronto transazionale della cardinalità del grafo prima e dopo la rimozione per confermare l'esito senza violare l'Information Hiding.

6.2.4. `showSeg`

Pre-condizioni: `root` valido, non nullo e precedentemente popolato.

Post-condizioni: I primi 20 record dell'indice cronologico vengono impaginati nella griglia geometrica ed emessi su standard output. Lo stato della memoria RAM e la disposizione dei collegamenti ortogonali rimangono inalterati.

Effetti collaterali: Blocca temporaneamente l'esecuzione tramite `getchar()` in attesa del comando utente.

6.2.5. `init_search_seg`

Pre-condizioni: `root` valido e popolato.

Post-condizioni: Al termine della routine, il terminale viene ripristinato nello stato originario. Il grafo non subisce modifiche.

Effetti collaterali: Altera temporaneamente i flag della TTY POSIX tramite `system("stty -icanon -echo")` per intercettare i singoli caratteri in tempo reale. Ripristina la modalità canonica prima del rientro.

6.2.6. `modifySegHud`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: Se l'utente preme ESC (codice 27), interrompe la funzione eseguendo

un rollback immediato senza mutare la memoria. Se l'input è valido, delega la transizione all'ADT tramite `modifySeg()`. Il terminale viene ripristinato prima del rientro.

Effetti collaterali: Modifica temporanea dei flag del terminale; emissione di alert grafici a schermo in base al codice di ritorno di `modifySeg()` ($-1, 0, 1$).

6.2.7. `showDescription`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: Rintraccia il nodo nel grafo tramite `findSegByID()` ed emette il dettaglio informativo a video. Lo stato della RAM rimane invariato.

Effetti collaterali: Invoca `getData()` e libera il buffer restituito tramite `free()` prima dell'uscita, garantendo l'assenza di memory leak.

6.2.8. `salvataggio`

Pre-condizioni: `sistema` valido e non nullo.

Post-condizioni: I dati attuali della RAM vengono interamente persistiti su disco tramite `save_records()`. Il report statistico viene aggiornato tramite `getReport()`. Tutta la memoria dinamica dell'Heap viene deallocata tramite `deleteGraph()`.

Effetti collaterali: Questa funzione interrompe il processo invocando `exit(0)` e non ritorna mai al chiamante. Visualizza una barra di caricamento animata tramite `usleep()`.

6.2.9. `intestation (interna)`

Pre-condizioni: Il terminale deve supportare la codifica ASCII standard.

Post-condizioni: I delimitatori geometrici e i titoli delle colonne della griglia tabellare vengono emessi su `stdout`. Nessuna allocazione di memoria dinamica viene effettuata.

Effetti collaterali: Scrittura su standard output.

6.2.10. `getReport (interna)`

Pre-condizioni: `sistema` valido e non nullo.

Post-condizioni: Un file di testo formattato viene generato o sovrascritto nella cartella `src/` con il nome `report_comune.txt`. Lo stato della memoria RAM rimane completamente inalterato.

Effetti collaterali: Apertura e scrittura su file stream in modalità "w" (distruttiva). Il file stream viene correttamente chiuso prima dell'uscita. Interroga l'ADT esclusivamente tramite i getter pubblici, rispettando l'Information Hiding.

6.2.11. `getSeg (interna)`

Pre-condizioni: node valido, non nullo e correttamente allocato nel grafo.

Post-condizioni: I dati del record vengono formattati in una stringa monospazio ed emessi su standard output. Il buffer temporaneo della data allocato sullo Heap viene interamente deallocato tramite `free()`.

Effetti collaterali: Invoca `getData()` che esegue un'allocazione dinamica; acquisisce la titolarità del puntatore ed esegue la `free()` prima dell'uscita.

6.3. Specifica sintattica di `struct.c`

Le funzioni **pubbliche** esportate da `struct.h` sono:

```
Root      init_root(void);
void      init_loadingDb(Root r, const char *fileName);
void      init_sorting(Root r);
void      init_removeSeg(Root r, int32_t idTarget);
int       getTotalSeg(Root root);
int       getTotalAperte(Root root);
int       getTotalRis(Root root);
int       getTotalChiuse(Root root);
int       getMostUrgenti(Root root);
s         getDataHead(Root root);
int       getSegCountByCategory(Root root, int catIdx);
int       getMaxCat(Root root);
const char *getMaxCatName(Root root);
int32_t   getID(s node);
const char *getName(s node);
const char *getDescription(s node);
const char *getCat(s node);
int       getRawData(s node);
int       getUrg(s node);
int       getState(s node);
char      *getData(s node);
s         nextForID(s node);
s         nextForData(s node);
s         nextForUrg(s node);
void      getNewSeg(Root root);
void      save_records(Root r);
void      deleteGraph(Root root);
void      search_seg(Root r, const char *searchString);
int       modifySeg(Root root, int32_t currId, int newState);
s         findSegByID(Root root, int32_t idTarget);
```


Le funzioni **interne** (non esportate, confinate in `struct.c`) sono:

```
static int    getCategoryIndex(int prefisso);
static bool   insertOnGraph(Root r, s newSeg);
void          quicksort(s *arr, int low, int high, int type);
int32_t       getRandomId(Root root, int prefix, int catIdx);
void          getPosition(Root root, s newSeg, int catIdx);
void          appendNewSeg(s newSeg, const char *fileName);
```

6.4. Specifica semantica di `struct.c`

6.4.1. `init_root`

Pre-condizioni: Nessuna.

Post-condizioni: Alloca la struttura principale `Root` e l'intera matrice dei descrittori di primo e secondo livello. Tutti i puntatori interni e i contatori vengono azzerati bit a bit tramite `calloc()`.

Return: Puntatore `Root` valido su Heap; `NULL` se l'allocazione fallisce.

Effetti collaterali: Riserva spazio di memoria dinamica sulla Heap.

6.4.2. `init_loadingDb`

Pre-condizioni: `r` valido e non nullo; `fileName` stringa non nulla che identifica un file binario esistente e accessibile in lettura.

Post-condizioni: Il grafo multi-indice viene espanso incorporando esclusivamente i record binari validi e integri. Il file stream viene correttamente chiuso. In caso di record parziale (lettura incompleta), il nodo parziale viene deallocato tramite `free()` prima di interrompere il ciclo.

Effetti collaterali: Apertura di un file stream in modalità "`rb`"; allocazioni dinamiche tramite `calloc()` per ciascun nodo valido; incremento dei contatori interni.

6.4.3. `init_sorting`

Pre-condizioni: `r` valido, non nullo e contenente un volume di record coerente con `totSegnalazioni`.

Post-condizioni: L'intera struttura viene temporaneamente linearizzata in un vettore di supporto. I quattro indici ortogonali della RAM vengono rigenerati: l'indice cronologico risulta ordinato per data crescente; i bucket delle categorie, dello stato e dell'urgenza vengono ricatenati in ordine crescente. Il vettore di supporto viene deallocato tramite `free()` prima dell'uscita.

Effetti collaterali: Allocazione temporanea di un array di puntatori; chiamate ricorsive a `quicksort()`; azzeramento e ricostruzione delle teste di tutti i descrittori.

6.4.4. `init_removeSeg`

Pre-condizioni: `r` valido; `idTarget` intero il cui prefisso sia mappabile da `getCategoryIndex()`.

Post-condizioni: Se la segnalazione viene individuata, viene scollegata dai quattro assi del grafo, i contatori globali e parziali vengono decrementati e la memoria Heap del nodo viene liberata. Se il record non viene trovato, lo stato del sistema resta invariato e viene emesso un messaggio a video.

Effetti collaterali: Scansioni lineari isolate su ciascuna catena (`nextId`, `nextData`, `nextUrg`, `nextStato`) per rintracciare il predecessore e bypassare i puntatori del target.

6.4.5. Getter globali: `getTotalSeg`, `getTotalAperte`, `getTotalRis`, `getTotalChiuse`, `getMostUrgenti`, `getDataHead`

Pre-condizioni: `root` deve essere un'istanza valida allocata tramite `init_root()`.

Post-condizioni: Lo stato della memoria RAM non subisce alcuna alterazione.

Return: Tutti restituiscono il valore del contatore pre-calcolato corrispondente accedendovi in tempo costante $\mathcal{O}(1)$, oppure 0/NULL se `root` è nullo.

Effetti collaterali: Nessuno.

6.4.6. `getSegCountByCategory`

Pre-condizioni: `root` valido; `catIdx` $\in [0, \text{allCat} - 1]$.

Post-condizioni: Lo stato del grafo e dei contatori non subisce alcuna alterazione.

Return: Numero esatto di segnalazioni registrate nella categoria indicata, acceduto in $\mathcal{O}(1)$ tramite il vettore `nCat`; 0 se la radice è nulla o l'indice è fuori range.

Effetti collaterali: Nessuno.

6.4.7. `getMaxCat`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: Lo stato del grafo e dei vettori dei contatori non subisce alterazioni.

Return: Indice intero (0–10) della categoria più colpita dai disservizi, trovato in $\mathcal{O}(\text{allCat})$ scorrendo il vettore `nCat`; -1 se il sistema è vuoto.

Effetti collaterali: Nessuno.

6.4.8. `getMaxCatName`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: Lo stato del grafo in memoria RAM rimane invariato.

Return: Puntatore a stringa costante contenente il nome della categoria dominante; "N/D" se il sistema è vuoto o l'indice non è valido. La stringa risiede nel segmento dati del programma: nessuna allocazione dinamica.

Effetti collaterali: Nessuno.

6.4.9. Getter atomici: `getID`, `getName`, `getDescription`, `getCat`, `getRawData`, `getUrg`, `getState`

Pre-condizioni: `node` deve essere un record valido.

Post-condizioni: Lo stato interno del nodo non subisce alcuna alterazione.

Return: Ogni funzione restituisce il campo corrispondente del nodo in $\mathcal{O}(1)$, oppure un valore di guardia (0, "Not Found", NULL) se `node` è nullo.

Effetti collaterali: Nessuno. Nessuna allocazione di memoria dinamica.

6.4.10. `getData`

Pre-condizioni: `node` puntatore a un record valido.

Post-condizioni: Estrae il timestamp intero (AAAAMMGG) e lo formatta nella stringa "GG/MM/AAAA". Il nodo interno non subisce alterazioni.

Return: Stringa allocata dinamicamente di 16 byte; NULL se `node` è nullo o se l'allocazione fallisce.

Effetti collaterali: Allocazione di memoria Heap tramite `malloc()`. Il modulo chiamante acquisisce la responsabilità di invocare la `free()` sul puntatore restituito per evitare memory leak.

6.4.11. Iteratori: `nextForID`, `nextForData`, `nextForUrg`

Pre-condizioni: `node` deve essere inserito coerentemente nel grafo multi-indice.

Post-condizioni: Lo stato del nodo e del grafo rimane invariato.

Return: Puntatore al nodo logicamente successivo nel rispettivo asse ortogonale (`nextId`, `nextData`, `nextUrg`), acceduto in $\mathcal{O}(1)$; NULL se la catena è terminata o se `node` è nullo.

Effetti collaterali: Nessuno.

6.4.12. `getNewSeg`

Pre-condizioni: `root` valido e non nullo.

Post-condizioni: In caso di successo, un nuovo record univoco viene inserito nel grafo e serializzato in modalità append su disco tramite `appendNewSeg()`; tutti i contatori

di sistema vengono incrementati e il grafo viene riordinato tramite `init_sorting()`. In caso di input non valido, il nodo parzialmente allocato viene liberato tramite `free()` e lo stato del grafo rimane invariato.

Effetti collaterali: Interazione con `stdin` per l'acquisizione interattiva dei dati. Generazione automatica di ID univoco tramite `getRandomId()`, data tramite le primitive POSIX `time()` e `localtime()`. Scrittura su file binario in modalità append.

6.4.13. `save_records`

Pre-condizioni: `r` valido, non nullo e coerentemente popolato.

Post-condizioni: Il file `database.bin` viene interamente sovrascritto con i dati aggiornati del grafo, scorrendo l'asse cronologico per preservare l'ordine nativo. Lo stato dei nodi in memoria Heap rimane inalterato.

Effetti collaterali: Apertura del file in modalità `"wb"` (distruttiva); scrittura sequenziale dei campi fissi di ciascun nodo; chiusura del file stream prima dell'uscita.

6.4.14. `deleteGraph`

Pre-condizioni: `root` puntatore valido (gestisce anche il caso di radice nulla o parzialmente allocata).

Post-condizioni: Tutta la memoria Heap allocata per i nodi segnalazione e per le strutture di controllo viene liberata. Nessun blocco rimane orfano.

Effetti collaterali: Deallocazione dell'intero Heap tramite scorrimento esclusivo dell'asse cronologico (`nextData`), seguito dalla liberazione in sequenza di tutti i descrittori di primo e secondo livello. Il puntatore del chiamante diventa invalido (*Dangling Pointer*).

6.4.15. `search_seg`

Pre-condizioni: `r` valido; `searchString` non nulla.

Post-condizioni: Lo stato del grafo in memoria RAM non subisce alcuna alterazione. I record individuati vengono formattati e stampati su standard output; le righe vuote residue vengono riempite geometricamente per preservare l'allineamento dell'HUD.

Effetti collaterali: Partizione adattiva del canale di navigazione: se il prefisso corrisponde a una categoria valida, la scansione è limitata al bucket (`nextId`); altrimenti avviene sull'intero indice cronologico (`nextData`).

6.4.16. `modifySeg`

Pre-condizioni: `root` non nullo; `newState` $\in [0, 2]$.

Post-condizioni: Se l'ID esiste e lo stato è differente, il nodo viene scollegato dalla vecchia sotto-lista di stato (splicing lineare sul canale `nextStato`) e inserito in testa

alla nuova, aggiornando i contatori atomici. Se l'ID non esiste o lo stato è già quello richiesto, la RAM resta invariata.

Return: 1 transazione completata; 0 stato già conforme; -1 ID non presente o parametri fuori range.

Effetti collaterali: Modifica dei puntatori interni `nextStato` del nodo bersaglio e dei nodi adiacenti; aggiornamento dei contatori di stato.

6.4.17. `findSegByID`

Pre-condizioni: `r` valido; `idTarget` intero a 32 bit il cui prefisso sia mappabile da `getCategoryIndex()`.

Post-condizioni: Lo stato del grafo rimane invariato.

Return: Puntatore opaco al nodo trovato; `NULL` se l'ID non esiste o la radice è nulla.

Effetti collaterali: Scansione lineare sul canale `nextId` del bucket della categoria corrispondente.

6.4.18. `getCategoryIndex` (interna)

Pre-condizioni: prefisso intero estratto come le prime due cifre dell'ID.

Post-condizioni: Lo stato del sistema rimane invariato.

Return: Indice intero (0–10) corrispondente alla cella del vettore della categoria; -1 se il prefisso non appartiene a nessuna classe censita.

Effetti collaterali: Nessuno.

6.4.19. `insertOnGraph` (interna)

Pre-condizioni: `r` e `newSeg` non nulli; `newSeg` completamente valorizzato.

Post-condizioni: Il nodo viene agganciato in testa in $\mathcal{O}(1)$ alle quattro liste dei descrittori ortogonali. I contatori parziali e globali vengono incrementati di 1 unità.

Return: `true` se il record viene agganciato con successo; `false` se l'indice di categoria non è valido.

Effetti collaterali: Modifica dei puntatori di testa di ciascun descrittore. Usato esclusivamente durante il caricamento da disco: l'allineamento topologico finale è delegato a `init_sorting()`.

6.4.20. quicksort (interna)

Pre-condizioni: `arr` array pre-allocato; `low` e `high` indici rientranti nei limiti fisici del vettore.

Post-condizioni: Gli indirizzi di memoria contenuti in `arr` vengono riordinati in-place. I dati fisici dei nodi non vengono copiati: vengono scambiati solo i puntatori.

Parametro type: 1 = Data, 2 = ID, 3 = Urgenza, 4 = Stato.

Effetti collaterali: Ricorsione su sottovettori sinistro e destro rispetto al pivot. Complessità media $\mathcal{O}(N \log N)$.

6.4.21. getRandomId (interna)

Pre-condizioni: `root` valido; `catIdx` $\in [0, \text{allCat} - 1]$; `prefix` codice ministeriale valido a due cifre.

Post-condizioni: Lo stato del grafo rimane invariato.

Return: Identificativo intero a 32 bit univoco all'interno del sistema corrente.

Effetti collaterali: Ciclo `do-while` difensivo: se viene generato un ID duplicato, l'algoritmo rigenera finché non trova un valore univoco, scansionando linearmente il bucket della categoria.

6.4.22. getPosition (interna)

Pre-condizioni: `root` e `newSeg` non nulli; `catIdx` $\in [0, \text{allCat} - 1]$.

Post-condizioni: Il nodo viene inserito in posizione ordinata negli assi `nextData` e `nextId` (inserimento ordinato con scansione lineare), e in testa in $\mathcal{O}(1)$ negli assi `nextStato` e `nextUrg`. Tutti i contatori vengono incrementati.

Effetti collaterali: Usata esclusivamente per l'inserimento a caldo di nuovi record tramite `getNewSeg()`.

6.4.23. appendNewSeg (interna)

Pre-condizioni: `newSeg` valido e completamente valorizzato; `fileName` percorso accessibile in scrittura.

Post-condizioni: I campi del nodo vengono serializzati su disco in un blocco binario contiguo. Lo stream viene chiuso correttamente. Lo stato del grafo in memoria RAM rimane inalterato.

Effetti collaterali: Apertura del file in modalità `"ab"` (append binario), che garantisce l'aggiunta del record senza riscrivere i dati preesistenti.

6.5. Specifica sintattica di main.c

```
void init(void);  
void handle_signal(int signum);  
int main(void);
```

6.6. Specifica semantica di main.c

6.6.1. init

Pre-condizioni: Il file database binario deve esistere ed essere accessibile.

Post-condizioni: La struttura Root viene allocata e inizializzata; le segnalazioni vengono caricate dal file binario; gli indici ortogonali vengono ricostruiti e ordinati; il tempo di caricamento viene misurato e visualizzato a video.

Effetti collaterali: Popola la variabile globale `sistema`; utilizza le primitive POSIX `clock()` per la misurazione delle prestazioni.

6.6.2. handle_signal

Pre-condizioni: Intercettazione del segnale SIGINT (Ctrl+C).

Post-condizioni: Interrompe in sicurezza l'applicazione evitando la terminazione sporca del processo.

Effetti collaterali: Invoca `salvataggio(sistema)`, che a sua volta persiste il database, dealloca il grafo e chiama `exit(0)`.

7. Suite di unit testing: main.t.c

Il file `src/test/main.t.c`, accennato nella panoramica dei moduli (Sezione 4.3.3), costituisce la corazzatura di isolamento del progetto. È una suite di unit testing pura che non interagisce con le funzioni grafiche dell'HUD, concepita per verificare in modo matematico la correttezza dell'ADT mediante le macro di asserzione strutturale fornite da `<assert.h>`.

Le asserzioni agiscono come sbarramenti logici invalicabili: se una singola funzione restituisce un valore non conforme alle specifiche semantiche, l'asserzione fallisce interrompendo istantaneamente l'esecuzione del programma e stampando la riga esatta del sorgente in cui si è verificata l'anomalia.

L'ispezione analitica ripercorre la suite test per test:

- **Inizializzazione e Bootstrap:** Il test invoca `init_root()` e applica immediatamente lo sbarramento `assert(test_root != NULL)`. Questo accerta che la struttura di controllo principale sia allocata correttamente in Heap prima di eseguire qualsiasi operazione sul database. Successivamente, tramite `init_loadingDb` e `init_sorting`, viene convalidata l'operazione di streaming da disco e la corretta edificazione iniziale della topologia del grafo multi-indice, verificando con `assert(totIniziale > 0)` il

caricamento dei record.

- **Test 1 — Consistenza dei Contatori Aggregati:** Dopo il popolamento del sistema, il test estrae i parziali degli stati operativi e verifica l'invariante di consistenza strutturale:

```
int aperte = getTotalAperte(test_root);
int risol  = getTotalRis(test_root);
int chiuse = getTotalChiuse(test_root);
assert(aperte + risol + chiuse == getTotalSeg(test_root));
```

Questo garantisce che l'inserimento nei quattro indici sia avvenuto in modo simmetrico, che non vi siano elementi orfani o duplicati nelle catene operazionali e che i contatori interni non siano corrotti.

- **Test 2 — Robustezza del Motore di Ricerca e Casi Limite:** Viene testata la stabilità e la sicurezza difensiva della funzione `search_seg` sottoponendola a sollecitazioni di input non standard, spurie o fuori range:

```
search_seg(test_root, "");
search_seg(test_root, "-999");
search_seg(test_root, "99999999");
```

Il test convalida che stringhe vuote, identificativi negativi o chiavi numeriche sovrabbondanti rispetto al range di targa vengano intercettati dal modulo senza innescare violazioni di memoria (Segmentation Fault) o cicli infiniti. Inoltre, verifica che la funzione effettui correttamente il partizionamento dinamico della scansione quando rileva un prefisso valido (es. "11" o "10"), restringendo la ricerca al singolo bucket di categoria.

- **Test 3 — Verifica dei Getter Statistici e della Categoria Dominante:** Il test convalida le funzioni dedicate all'estrazione di metriche aggregate. Viene accertato che il contatore delle massime urgenze rispetti i confini di senso (`assert(urgenti >= 0 && utenti <= totIniziale)`). Successivamente, viene isolato l'indice della categoria più colpita tramite `getMaxCat` e `getMaxCatName`, applicando un rigoroso controllo comparativo ad albero:

```
int maxIdx = getMaxCat(test_root);
assert(maxIdx >= 0 && maxIdx <= 10);
int maxCount = getSegCountByCategory(test_root, maxIdx);
for (int i = 0; i <= 10; i++) {
    assert(getSegCountByCategory(test_root, i) <= maxCount);
}
```

Tale routine matematica certifica che nessuna categoria nel sistema possieda una densità di record superiore a quella identificata come massima assoluta dal modulo statistico dell'ADT.

- **Test 4 — Validazione dell’Attraversamento Cronologico:** Il test estrae la testa cronologica (`getDataHead`) e naviga i primi 20 nodi usando esclusivamente l’iteratore `nextForData`. Ad ogni iterazione applica asserzioni rigide: `getID(curr) != 0`, `getRawData(curr) > 0`, il range di urgenza `[1, 5]` e la conformità dello stato operativo `[0, 2]`. Questo accerta che l’asse cronologico sia perfettamente inanellato e non contenga puntatori orfani o dati inconsistenti.
- **Test 5 — Controllo della Responsabilità di Memoria di `getData`:** Verifica che `getData(NULL) == NULL` per confermare la robustezza del codice difensivo a sbarramento preventivo. Su un nodo valido, accerta che la stringa alfanumerica restituita contenga la corretta formattazione standard (`GG/MM/AAAA`) con lunghezza esattamente pari a 10 caratteri e che il ciclo di allocazione dinamica sia simmetrico e privo di leak sotto Valgrind.
- **Test 6 — Validazione Atomica delle Transazioni di Stato:** Testa tutti i sentieri logici e i codici di ritorno della macchina a stati implementata in `modifySeg`:

```
assert(modifySeg(test_root, targetId, nuovoStato) == 1);
assert(modifySeg(test_root, targetId, nuovoStato) == 0);
assert(modifySeg(test_root, 9999999, 1) == -1);
assert(modifySeg(test_root, targetId, 99) == -1);
```

Questo test valida matematicamente il comportamento deterministico dello splicing ortogonale dei puntatori: il valore 1 per una transizione completata, 0 per una transizione ridondante sullo stesso stato e -1 per violazioni di range dello stato o ID inesistenti. Viene inoltre monitorato il contatore globale per assicurare che rimanga costante durante i riposizionamenti di coda.

- **Test 7 — Efficacia della Ricerca Puntuale del Nodo:** Viene validata la funzione d’interfaccia `findSegByID`. Il test verifica che la scomposizione matematica dell’ID e il successivo scorrimento mirato sul solo bucket della categoria associata restituiscano il puntatore corretto all’indirizzo fisico del nodo opaco se l’ID esiste (`assert(getID(trovato) == targetId)`), ed emetta in sicurezza un puntatore nullo (`NULL`) qualora si interroghi il sistema con chiavi non censite.
- **Test 8 — Integrità e Irrintracciabilità post-rimozione:** Dopo la chiamata a `init_removeSeg`, il test verifica l’estirpazione definitiva del record rintracciato:

```
assert(getTotalSeg(test_root) == totPrima - 1);
assert(findSegByID(test_root, targetId) == NULL);
```

Questo certifica che il nodo sia stato completamente sfilato da tutti e quattro gli indici ortogonali del grafo (categoria, timeline, stato, urgenza), che le adiacenze dei nodi attigui siano state ricucite in modo transazionale e che la chiamata difensiva su ID inesistenti non provochi mutazioni o instabilità nei contatori o nella RAM.

- **Test 9 — Verifica della Persistenza Informativa del Report:** Viene esaminata la capacità del sistema di interagire con il file system per la scrittura dei dati di sintesi. Il test convalida che l'apertura del report testuale in modalità distruttiva ("**w**") avvenga con successo e che la riscrittura delle metriche rispecchi fedelmente la densità informativa corrente del grafo prima dello shutdown.

Al termine della suite di test, la chiamata esplicita a `deleteGraph(test_root)` esegue il cleanup totale dello Heap, convalidando le politiche di deallocazione controllata che consentono al software di superare i test di profiling di Valgrind con zero leak.

8. Ambiente di sviluppo e toolchain

L'intero ecosistema software è stato sviluppato, ottimizzato e testato per operare in ambienti nativi UNIX-like ad alte prestazioni.

8.1. Sistema operativo

Il sistema operativo utilizzato per lo sviluppo è **Arch Linux**, una distribuzione rolling-release orientata ai programmatori che favorisce il controllo crudo del sistema. L'applicazione è stata concepita per girare in modalità testuale nativa all'interno di una **TTY pura**, con l'interfaccia grafica rimossa dal boot manager tramite `systemctl`. Questo ha permesso di testare il software in un ambiente minimale, privo di filtri grafici, esaltando la nitidezza dell'HUD geometrico e la velocità di risposta dei flag POSIX.

8.2. Compilatore

La compilazione è affidata a **GCC (GNU Compiler Collection)**, versione 15.1.0, utilizzato per la stabilità nell'ottimizzazione del codice macchina e nel tracciamento dei vettori di puntatori.

8.3. Standard C

Il codice rispetta rigorosamente lo standard **C99** (`-std=c99`). Questa specifica introduce i tipi interi a dimensione fissa definiti in `<stdint.h>` (come `int32_t`), fondamentali per garantire l'esatta dimensione dei record binari su disco indipendentemente dall'architettura hardware, e consente la dichiarazione delle variabili all'interno dei cicli `for`, mantenendo il codice pulito e moderno.

8.4. Flag di compilazione

I flag impostati nel Makefile impongono sbarramenti difensivi precisi:

- `-Wall -Wextra`: Attivano la totalità dei controlli sintattici del compilatore, segnalando confronti tra tipi diversi o variabili inutilizzate.
- `-O2`: Abilita l'ottimizzazione a livello di registri della CPU, velocizzando i cicli interni di Quicksort e i riallineamenti dei puntatori.

- `-g`: Include i simboli di debug nel binario, mappando le istruzioni macchina con le righe del sorgente C per l'analisi con Valgrind e GDB.
- `-D_DEFAULT_SOURCE`: Abilita l'esposizione delle macro POSIX standard necessarie alla manipolazione dei descrittori del terminale.

8.5. Sistema di build

La gestione delle dipendenze e l'automazione della compilazione strutturata sono governate da **GNU Make**.

8.6. Analisi dinamica della memoria

Il monitoraggio della stabilità dell'Heap è affidato a **Valgrind 3.25.1 (Memcheck Subsystem)**, utilizzato per verificare l'assenza di leak o sovrascritture di buffer.

8.7. Analisi di debug a runtime

L'ispezione a runtime degli stati interni delle strutture opache è gestita tramite **GDB (GNU Debugger)**.